Informe Gerencial — Proyecto FleetIQ

Oct 7, 2026 · @George

Se desarrolló y validó **FleetIQ**, una aplicación web local (archivo HTML único) de analítica de mantenimiento y confiabilidad de flotas, construida a partir de un prompt maestro de 45 secciones y sometida a pruebas de carga que revelaron y corrigieron tres fallas técnicas antes de su entrega final.

1. Alcance y objetivo del proyecto

El objetivo fue construir una herramienta profesional de **Fleet Maintenance Analytics**, entregada como un único archivo `dashboard.html` que funciona sin servidor, sin conexión a internet y sin exponer datos a terceros. A diferencia de un dashboard fijo, la aplicación debía adaptarse a datasets desconocidos: detectar automáticamente su estructura, mapear nombres de columna heterogéneos a variables de ingeniería estándar, y calcular solo los indicadores que las variables disponibles permitan — sin inventar cifras cuando falte información.

El desarrollo se basó en una especificación técnica de 45 secciones (rol, arquitectura modular, importación, perfilado, mapeo semántico, motores de analítica, KPI, visualización, filtros, insights, calidad de datos y exportación).

2. Capacidades principales entregadas

Módulo	Función
Importación	Lee CSV, XLSX, XLS, JSON y TXT delimitado directamente en el navegador (PapaParse y SheetJS embebidos, sin dependencias externas en tiempo de ejecución)
Perfilado de datos	Detecta automáticamente tipo de cada columna (numérica, fecha, categórica, identificador), nulos, duplicados y rangos

Módulo	Función
Mapeo semántico	Reconoce variables equivalentes bajo distintos nombres (p. ej. `EQ_EQUIP_NO`, `Vehicle_ID`, `Unidad` → vehículo) mediante un diccionario de sinónimos en español e inglés
Calidad de datos	Clasifica el dataset como Excelente / Buena / Aceptable / Deficiente con un puntaje 0–100 basado en criterios explícitos (nulos, duplicados, fechas inválidas, columnas constantes)
KPI de flota y confiabilidad	MTBF, MTTR, disponibilidad, tasa de fallas, costo por km/hora, cumplimiento preventivo — cada uno con su fórmula visible y con aviso explícito de "No disponible" cuando faltan variables
Visualización	Tendencias temporales, Pareto 80/20, ranking de vehículos, dispersión costo-kilometraje, detección de outliers (IQR)
Insights automáticos	Observaciones generadas por reglas con umbrales explícitos, separando dato observado de interpretación — nunca atribuye causalidad a una correlación
Filtros y exploración	Filtros cruzados (fecha, vehículo, tipo, sistema, costo), tabla con orden/búsqueda/paginación, exportación a CSV e impresión a PDF
Privacidad	Procesamiento 100% local; ningún dato del archivo cargado sale del navegador del usuario

3. Hallazgos relevantes durante la validación técnica

Al probar el dashboard con un archivo real de órdenes de trabajo (formato típico de sistema CMMS, con columnas como `UNIQUE_WORK_ORDER_NO`, `EQ_EQUIP_NO`, `TOTAL_COST`), la carga falló con el error `RangeError: Maximum call stack size exceeded`. La investigación — reproducida con un dataset sintético de 20.000 filas con la misma estructura — identificó tres fallas, ya corregidas:

#	Falla detectada	Causa	Corrección aplicada
1	Caída total de la aplicación con archivos grandes	Cálculo de mínimos/máximos con `Math.min(...arreglo)`, que excede el límite de argumentos del motor JavaScript en arreglos de miles de valores	Reemplazo por recorrido manual sin límite de tamaño
2	Columna de año (`WORK_ORDER_YR = "2.023"`) clasificada erróneamente como fecha	`new Date("2.023")` es interpretado por JavaScript como una fecha válida (23-feb-2001) en lugar de ser rechazado	Validación que descarta como fecha cualquier valor puramente numérico antes de intentar convertirlo
3	Variable de costo mal identificada (`LABOR_COST` en vez de `TOTAL_COST`)	El mapeo semántico priorizaba la columna por su posición en el archivo, no por la especificidad del nombre	Mapeo corregido para priorizar por el orden de especificidad del diccionario semántico, no por el orden del archivo

Tras las tres correcciones, el flujo completo (importación → perfilado → mapeo semántico → cálculo de calidad → limpieza) se validó sin errores sobre 20.000 filas sintéticas con la estructura real del cliente.

4. Estado actual y calidad del entregable

El archivo `dashboard.html` (≈1.2 MB, con todas sus librerías embebidas para funcionamiento 100% offline) está terminado y validado. Con la estructura de columnas del cliente (sistema CMMS tipo `UNIQUE_WORK_ORDER_NO` / `EQ_EQUIP_NO`), el mapeo semántico reconoce correctamente: vehículo, tipo de mantenimiento, costo, horas de operación, tiempo de reparación y fecha, lo que habilita el cálculo de MTBF, MTTR, disponibilidad y costo por hora sin intervención manual.

Una condición queda pendiente de confirmación del usuario y se registra como comentario en este documento: si `METER_1_READING` corresponde a kilometraje (supuesto actual) o a horómetro, según el tipo de equipo de la flota.

5. Recomendaciones y próximos pasos

1. Cargar el archivo real completo y revisar la pestaña "Acerca del análisis" del dashboard para confirmar el mapeo semántico aplicado a cada columna.
2. Confirmar si `METER_1_READING` representa kilometraje u horas de operación, y si la flota mezcla equipos con distinto tipo de medidor (en ese caso, conviene segmentar el análisis por `JOB_TYPE` o por `DEPT_EQUIP_DEPT_NAME` para evitar comparar unidades no homogéneas).
3. Si el volumen de registros supera ~30.000 filas de forma habitual, evaluar incorporar Web Workers y virtualización de tabla para mantener la fluidez de la interfaz.
4. Como extensión futura ya prevista en la arquitectura, evaluar un módulo de mantenimiento predictivo (análisis de Weibull/supervivencia) una vez se disponga de series históricas suficientes.